

Exercise 6.1 Graph Traversal

Consider the following graph G with root node A :

$A \rightarrow B$
 $B \rightarrow C$
 $C \rightarrow D$
 $C \rightarrow E$
 $D \rightarrow F$
 $E \rightarrow F$
 $F \rightarrow C$
 $C \rightarrow G$
 $G \rightarrow B$
 $B \rightarrow H$

1. Draw a graph from the given edges.
2. Give a valid preorder of all nodes.
3. Give a valid postorder of all nodes.
4. Give a valid reverse postorder of all nodes.
5. Give an algorithm that puts a preorder/postorder time stamp on all nodes within a graph *without* relying on a global/class counter variable. In other words, the signature of your DFS traversal should look like this:

```
int dfs(std::unordered_set<const Node*>& visited, const Node* n, int time);
```

You can also use other programming languages, if you like.

Exercise 6.2 CFG, Dom, Dom Frontier, Postdom & Postdom Frontier

Consider the following fragment of a C/Java program:

```

A;
while (B && C) {
    if (D || E) {
        F;
        continue;
    }
    G;
    if (H || I) break;
    J;
}
K;
    
```

1. Construct the CFG from this program.
2. Eliminate any critical edges that might occur in your version.
3. Compute the dominance tree.
4. Compute the postdominance tree.

5. Construct the reverse CFG from (1) by reverting all edges, using K as entry, and A as exit.
6. Compute the dominance frontier graph.
7. Compute the postdominance frontier graph (aka control dependence graph).

Exercise 6.3 Dominance is a partial order

Prove that the dominance relation is a partial order. To this end, you need to show that it is

- reflexive,
- transitive,
- and anti-symmetric.